

Lecture 19: Optimization Problems and Linear Programming

*Instructor: Sourav Chakraborty**Scribe: Anubhab Neogi, Rupraj Roy*

1 The Stable Matching Problem

Before getting into linear programming, let us look at a classic optimization scenario: the Stable Matching problem. Imagine an equal-sized set of boys and girls. Every boy has a ranked preference list of all the girls, and every girl has a ranked list of all the boys. The goal is to pair them up so that no boy and girl would secretly rather be with each other than their assigned partners.

To solve this, we use the Gale-Shapley algorithm, which follows a straightforward greedy approach:

- **Proposing:** In each round, every unengaged boy proposes to the top girl on his list who has not rejected him. He acts greedily by always trying for his favorite remaining option.
- **Evaluating:** Each girl looks at the proposals she just received, along with whoever she is currently engaged to. She temporarily accepts the proposal from the boy she ranks highest out of that specific group and rejects all the others.
- **Repeating:** The rejected boys move down to the next girl on their list in the next round. This repeats until everyone is paired up.

Let us prove why the Gale-Shapley algorithm is mathematically guaranteed to result in a stable matching. The proof is broken down into three logical steps: showing that the process finishes, showing that everyone gets a partner, and finally showing that the resulting pairs are truly stable.

Step 1: The algorithm always finishes (Termination)

There are n boys and n girls. Every time a boy makes a proposal, he crosses that girl's name off his list forever. He will never propose to the same girl twice. Since there are a total of $n \times n$ possible proposals, the algorithm must eventually stop. It cannot run in an infinite loop.

Step 2: Everyone gets paired up (Perfect Matching)

We prove this using contradiction. Suppose the algorithm finishes, but there is a boy, let's call him Bob, who is left without a partner.

If Bob is unmatched at the end, it means he must have gone down his entire list, proposed to every single girl, and been rejected by all of them. But what are the rules for a girl rejecting a boy? A girl only rejects a proposal if she already has a partner she likes better. Once a girl gets a partner, she stays engaged until the end (she only "trades up").

Therefore, if all n girls rejected Bob, it means all n girls are currently engaged to someone else. But there are exactly n boys and n girls. If all n girls are engaged, all n boys must be engaged. Bob cannot be left out. This is a contradiction, proving that everyone gets perfectly paired.

Step 3: The pairs are stable (No Rogue Couples)

Again, we use contradiction. Suppose the final matching is unstable. This means there is a boy (Bob) and a girl (Alice) who are not paired with each other, but Bob prefers Alice over his final partner, and Alice prefers Bob over her final partner.

Let's look at Bob. Since Bob prefers Alice to his final partner, and boys propose strictly from the top of their list downwards, Bob *must* have proposed to Alice before he proposed to his final partner.

Since Bob did not end up with Alice, it means Alice rejected Bob at some point. Why would Alice reject Bob? A girl only rejects a boy if she is holding a proposal from someone she prefers more. Because girls only trade up throughout the algorithm, Alice's final partner must be someone she ranks *higher* than Bob.

But wait! Our initial assumption was that Alice prefers Bob over her final partner. This is a direct contradiction. Therefore, no such unstable pair can exist. The matching is 100% stable.

The Gale-Shapley yields the best possible stable matching for the proposers and the worst for the receivers. This occurs due to the mechanics of the algorithm:

- **Proposers Play Offense (Top-Down):** Proposers start at the absolute top of their preference lists and only move down when rejected. This guarantees that a proposer is never rejected by a valid stable partner, ensuring they secure their highest-ranked possible match.
- **Receivers Play Defense (Bottom-Up):** Receivers are completely passive. They must wait for offers and can only “trade up.” Because they accept the first valid offer and wait to see if anyone better comes along, they are working upward from their worst-case scenario.
- **The Lattice Structure:** The set of all possible stable matchings forms a mathematical lattice. Because the algorithm pushes the proposers to the “ceiling” (the absolute top of the lattice), the strictly opposed interests of the two groups force the receivers to the “floor” (the bottom of the lattice), leaving them with their lowest-ranked valid partners.

While finding a single stable matching is fast and easy using this greedy method, generating a random stable matching with equal probabilities is a notoriously difficult open problem that relies on advanced randomized algorithms.

2 Optimization and Linear Programming

An optimization problem requires finding a solution that maximizes or minimizes a specific value, like cost or profit, while satisfying a set of given conditions. Common examples include finding the shortest path in a network, solving the Knapsack Problem, fitting a regression model, optimizing Support Vector Machines (SVM), or calculating the best k-means clustering.

Many optimization problems can be modeled as a Linear Program (LP). In an LP, the objective function and all the rules (constraints) are expressed as linear equations or inequalities. For instance, we might want to maximize $3x + 4y - 10z$ under constraints like $5x + 8y \leq 15$ and $z \geq 0$, where the variables belong to the real numbers ($\mathbb{R}$).

2.1 Complexity and Historical Algorithms

A critical property of Linear Programming is that it can be solved efficiently in polynomial time when the variables take real, continuous values. Historically, three major algorithmic milestones define how we solve LPs:

- **Simplex Method (1965):** Very simple to implement and highly efficient in normal, everyday cases. However, it does not guarantee polynomial time in the worst-case scenario.
- **Ellipsoid Method (1969):** The first algorithm to mathematically prove that LP is solvable in polynomial time. However, it is mathematically complex and often too slow for practical applications.
- **Interior Point Method (N. Karmarkar, 1982):** A massive breakthrough that is both simple to understand and highly efficient in true polynomial time. This method forms the backbone of many modern LP solvers.

While standard LP is solvable in polynomial time, Integer Linear Programming (ILP)—where variables must be whole numbers like 0 or 1—is a hard problem. We cannot expect

a fast, polynomial-time solution for ILP in the worst case.

2.2 The Modeling Hierarchy

When analyzing a new optimization problem, it is best to follow a specific modeling hierarchy:

1. **Linear Programming:** Try to model the problem as a standard LP first with real numbers, since it is easily solvable in polynomial time.
2. **Graphs:** If the standard LP does not fit naturally, try mapping the problem to intermediate structures like Graphs to visualize relationships.
3. **Integer Linear Programming (ILP):** If the problem requires strict discrete choices, model it as an ILP.
4. **Relaxed LP and Approximation:** Since ILP is hard, we relax the integer constraints back to real numbers ($0 \leq x \leq 1$) to obtain a Linear Program over Reals. We solve this relaxed version using software packages and then pull back the integer solution using approximation techniques.

3 Examples of Standard LP and ILP Modeling

3.1 Example 1: Transportation Optimization (Standard LP)

A trading company wants to maximize profit from transporting goods. They have 2 wagons and 3 types of cargo.

Wagons: W_1 (Weight: 10 tons, Space: 5000), W_2 (Weight: 8 tons, Space: 4000).

Cargo Details:

- C_1 : 18 tons available, takes 400 space/ton, yields 2000 profit/ton.
- C_2 : 10 tons available, takes 300 space/ton, yields 2500 profit/ton.
- C_3 : 5 tons available, takes 200 space/ton, yields 5000 profit/ton.

Let x_i be the amount of cargo loaded into W_1 , and y_i be the amount loaded into W_2 . We model this as an LP over real numbers:

$$\begin{aligned}
\text{Maximize} \quad & 2000(x_1 + y_1) + 2500(x_2 + y_2) + 5000(x_3 + y_3) \\
\text{Subject to} \quad & x_1 + y_1 \leq 18, \quad x_2 + y_2 \leq 10, \quad x_3 + y_3 \leq 5 \\
& x_1 + x_2 + x_3 \leq 10, \quad y_1 + y_2 + y_3 \leq 8 \\
& 400x_1 + 300x_2 + 200x_3 \leq 5000 \\
& 400y_1 + 300y_2 + 200y_3 \leq 4000 \\
& x_1, x_2, x_3, y_1, y_2, y_3 \geq 0 \text{ and } \in \mathbb{R}
\end{aligned}$$

Because the variables are real numbers, software libraries like ‘lpsolve’ can solve this quickly.

3.2 Example 2: Bang for Buck in Advertisement (ILP & Probabilistic Rounding)

You have a marketing budget of 6 Crore Rupees and only 10 staff members to handle campaigns. You must choose from 6 different advertising options.

Because you either select an ad campaign completely or not at all, the variables $x_1, \dots, x_6$ are boolean (0 or 1). This makes it an Integer Linear Programming problem.

$$\begin{aligned}
\text{Maximize} \quad & 10x_1 + 15x_2 + 0.5x_3 + 2.5x_4 + 4x_5 + 0.5x_6 \\
\text{Subject to} \quad & 3.5x_1 + 5.2x_2 + 0.1x_3 + 0.9x_4 + 1.5x_5 + 0.2x_6 \leq 6 \\
& 5x_1 + 7x_2 + 3x_3 + 4x_4 + 4x_5 + 2x_6 \leq 10 \\
& x_1, \dots, x_6 \in \{0, 1\}
\end{aligned}$$

To solve this mathematically hard problem, we use relaxation. We allow the variables to be real numbers $0 \leq x_i \leq 1$. We feed this relaxed LP into solvers like ‘lpsolve’ to get an

optimal continuous solution $p_1, \dots, p_6$.

We then map these decimals back to integers. One technique is to assign the final integer value $\hat{x}_i = 1$ with probability p_i , and 0 otherwise. This probabilistic rounding ensures our constraints and expected reach are satisfied on average.

4 Modeling via Graphs

Many real-world problems are easier to formulate if we first translate them into a Graph.

4.1 What is a Graph?

A graph is a simple, general mathematical way of expressing relations among objects. Formally, a graph $G = (V, E)$ consists of:

- **Vertices (V):** A set of elements $\{v_1, \dots, v_n\}$. These represent objects or states.
- **Edges (E):** A set of pairs of vertices $\{e_1, \dots, e_m\}$ where $e_k = (v_i, v_j)$. These represent relationships between the vertices.

Graphs can take on special properties depending on the problem:

- **Undirected Graphs:** Like a friendship graph for social networks. If A is friends with B, B is friends with A.
- **Directed Graphs:** Sometimes relations are not reflexive. If a website links to another website, the reverse isn't necessarily true. We add arrows to edges to show direction, which is how search engines crawl the internet.
- **Weighted Graphs:** Not all relationships hold the same importance or cost. For road networks or air/rail connectivity, we assign numerical weights (like distance or travel time) to the edges.

4.2 Example 3: Telephone Towers (Graph ILP & Thresholding)

There are N important locations in a city. A location gets a good 3G connection if it is less than 1km from an upgraded tower. The company wants to minimize its cost by upgrading as few towers as possible.

Graph Formulation: Let the vertices V be the combined set of important locations $v_1, \dots, v_n$ and tower locations $u_1, \dots, u_t$. We draw an edge between a location and a tower if the distance between them is less than 1km.

Objective: Pick a subset of the tower vertices such that every location vertex in the graph has at least one picked neighbor. We define variables $X_1, \dots, X_t$ representing whether a tower is chosen (1) or not (0).

$$\begin{aligned} &\text{Minimize} && \sum_i X_i \\ &\text{Subject to} && \text{For all vertex } v_j, \sum_{i|(u_i, v_j) \in E} X_i \geq 1 \\ &&& X_1, \dots, X_t \in \{0, 1\} \end{aligned}$$

Because this ILP is hard to solve, we relax the bounds to $0 \leq X_i \leq 1$ and solve the relaxed LP to obtain decimal values $p_1, \dots, p_t$. To convert these back to 0 or 1, we use a **Thresholding** technique: fix a threshold T (like 0.5), and set $X_i = 1$ if $p_i > T$, and 0 otherwise.

4.3 Example 4: Scheduling Meetings and Graph Coloring

Multiple meetings are scheduled with overlapping start and end times. Meetings with timing clashes cannot be held in the same room. We must calculate the minimum number of rooms required.

Graph Formulation: Every meeting is represented as a vertex. We draw an edge between two vertices if the two corresponding meetings have a time clash.

Objective: We need to slot meetings into different rooms so that if an edge exists between two vertices, they end up in different rooms. This is the exact definition of the **Vertex Coloring Problem**.

Given $G = (V, E)$, we want to color the vertices with k colors such that for all edges $(v_i, v_j) \in E$, the color of v_i is not equal to the color of v_j . The minimum number of colors required is called the **Chromatic Number** of the graph.

This identical concept applies to Map Coloring. If you take a political map of India, treat each state as a vertex, and draw an edge between states that share a border, you can use vertex coloring to ensure no two adjacent states are painted the same color.

Many problems in real life can be designed as problems in graph theory. By mapping a real-world scenario to vertices and edges, we can clearly define our constraints and translate the entire system into a Linear Program or an optimization problem. The real trick is simply modeling the problem in a mathematical way so we can apply these established computational techniques.